Cahier des Charges — Projet Python ING3 : Pipeline de Logs Sécurisé

Nov 5, 2026 · @mamadou

Projet : Engine de traitement et d'analyse de flux de données sécurisé en temps réel
Équipe : Mniangadou (Dev A) · Marlon (Dev B) · Baptiste (Dev C)
Durée : 4 semaines · **Langage :** Python 3.10+

1. Contexte et objectif

Concevoir une plateforme CLI Python 3.10+ d'ingestion, de filtrage, de transformation et d'analyse de logs système/réseau volumineux, sans jamais charger les fichiers en RAM.

Le projet est évalué sur **20 points** avec une pondération forte sur la gestion mémoire par générateurs (4 pts), les structures de données complexes (4 pts) et la programmation fonctionnelle (4 pts) — soit 12 points sur 20.

Contraintes techniques imposées par l'énoncé :

- Python 3.10+ obligatoire (nécessaire pour `match-case`).
- Environnement virtuel isolé (`venv` ou `conda`) avec `requirements.txt` fidèle (`pip freeze`). Aucune bibliothèque globale non listée ne sera tolérée.
- Code structuré en **package Python** (`.py` propres et exécutables). Les `.ipynb` sont réservés au prototypage et à la démo graphique finale.
- Un point d'entrée unique `main.py` pilote l'ensemble du pipeline.
- Lecture de logs **par générateurs** (`yield`, `yield from`), jamais par `read()` global.
- Respect PEP8, typage, gestion propre des exceptions, utilisation de la clause `else` des boucles.

2. Répartition des rôles

Chaque développeur possède un périmètre clair + une responsabilité transversale (pour équilibrer les charges, Module 1 étant très dense).

Développeur	Rôle	Modules pilotés	Fichiers livrés	Responsabilité transversale
Mniangadou — Dev A	Data engineer / ingestion	Module 1 (Ingestion & Générateurs)	`src/ingestion/log_generator.py`, `src/ingestion/parser.py`, `src/utils/types.py`	Jeu de données de test (`data/`), génération d'un fichier de logs synthétique volumineux (≥ 100 Mo), vérification RAM via `tracemalloc`

Développeur	Rôle	Modules pilotés	Fichiers livrés	Responsabilité transversale
Marlon — Dev B	Pipeline / programmation fonctionnelle	Module 2 (Pipeline fonctionnel)	`src/pipeline/filters.py`, `src/pipeline/transforms.py`, `src/pipeline/router.py`, `main.py`	Orchestration `main.py`, interface CLI (`argparse`), qualité code transversale (black, ruff, type hints)
Baptiste — Dev C	Analyse & sécurité	Module 3 (Agrégation) + Module 4 (Rule Engine)	`src/analytics/sets_ops.py`, `src/analytics/counters.py`, `src/analytics/bruteforce.py`, `src/engine/rule_engine.py`, `src/engine/audit.py`	Rapport final `notebooks/demo_rapport.ipynb` avec graphiques matplotlib, présentation soutenance

Tout le monde contribue également à : tests unitaires de sa zone (`tests/`), relecture (code review) des PR des autres, mise à jour du `README.md` .

3. Architecture cible du projet

Arborescence à créer en début de projet (semaine 1) et à respecter strictement par chaque développeur :

```
projetspython/
├── venv/                        # ignoré par git (.gitignore)
├── requirements.txt             # généré via pip freeze
├── README.md                    # install, usage CLI, structure
├── main.py                      # point d'entrée CLI (argparse)
├── src/
│   ├── __init__.py
│   ├── ingestion/               # Module 1 — Dev A
│   │   ├── __init__.py
│   │   ├── log_generator.py     # yield / yield from
│   │   └── parser.py            # parsing ligne -> dict + tuples
│   ├── pipeline/                # Module 2 — Dev B
│   │   ├── __init__.py
│   │   ├── filters.py           # filter() + lambda
│   │   ├── transforms.py        # map() + normalisation
│   │   └── router.py            # match-case
│   ├── analytics/               # Module 3 — Dev C
│   │   ├── __init__.py
│   │   ├── sets_ops.py          # set / frozenset / unions / intersections
│   │   ├── counters.py          # defaultdict
│   │   └── bruteforce.py        # while/for + break/continue/else
│   ├── engine/                  # Module 4 — Dev C
│   │   ├── __init__.py
│   │   ├── rule_engine.py       # *args, **kwargs, first-class fns
│   │   └── audit.py             # globals() / locals()
│   └── utils/                   # commun
│       ├── __init__.py
```

```
|       ├── logger.py            # logging centralisé
|       └── types.py             # TypedDict, enums sévérité, alias
├── data/
|   ├── sample_small.log         # ~1 Mo pour tests rapides
|   └── sample_large.log         # >= 100 Mo pour preuve non-RAM
├── notebooks/
|   └── demo_rapport.ipynb       # démo graphique + rapport final
└── tests/
    ├── __init__.py
    ├── test_ingestion.py
    ├── test_pipeline.py
    ├── test_analytics.py
    └── test_engine.py
```

Règle de nommage : fichiers et fonctions en `snake_case`, classes en `PascalCase`, constantes en `UPPER_SNAKE`. Un module = une responsabilité.

4. Spécifications détaillées par module

Module 1 — Ingestion & Générateurs mémoire (Dev A, 4 pts)

Objectif : lire des fichiers de logs volumineux sans faire exploser la RAM.

Fonctions clés attendues :

```
def read_log_file(path: str) -> Iterator[str]:
    """Générateur yield qui lit un fichier ligne par ligne."""

def read_multiple_sources(paths: list[str]) -> Iterator[str]:
    """Sous-générateur utilisant yield from pour chaîner plusieurs
fichiers."""

def parse_line(line: str) -> dict:
    """Parse une ligne en dict typé : timestamp, level, src_ip, user,
payload."""

# Métadonnées fixes immuables
ConnectionMeta = tuple[str, int, str]  # (IP, Port, Protocole)
```

Exigences :

- Utiliser **impérativement** `yield` dans `read_log_file` et **`yield from`** dans `read_multiple_sources`.

- Chaque ligne parsée retourne un dict avec les champs : `timestamp`, `level` (INFO/WARNING/ERROR/CRITICAL), `src_ip`, `user_id`, `payload` (JSON ou string brute).
- Représenter les métadonnées fixes avec des **tuples** (immuables).

Module 2 — Pipeline & Programmation Fonctionnelle (Dev B, 4 pts)

Objectif : chaîner filtrage, normalisation et routage sans boucles impératives.

Fonctions clés attendues :

```
def filter_invalid(logs: Iterator[dict]) -> Iterator[dict]:
    """filter() + lambda pour exclure les entrées invalides/vides."""
    return filter(lambda log: log and log.get('level'), logs)

def normalize(logs: Iterator[dict]) -> Iterator[dict]:
    """map() + f-strings pour normaliser casse, format timestamp, trim."""

def route_log(entry: str | dict | tuple) -> str:
    """match-case pour router selon le format : brute / dict / tuple
connexion."""
    match entry:
        case str(raw):
            return 'raw'
        case {'level': 'ERROR', 'code': code}:
            return f'error_{code}'
        case (ip, port, proto):
            return 'connection'
        case _:
            return 'unknown'
```

Exigences :

- `filter()` et `map()` obligatoires (interdiction d'utiliser des list comprehensions à la place pour cette partie).
- Au moins **3 cas distincts** dans le `match-case` (brute, dict, tuple).

Module 3 — Agrégation & Traitement avancé (Dev C, 4 pts partagés avec Module 4)

Fonctions clés attendues :

```
BLACKLIST: frozenset[str] = frozenset({'10.0.0.1', '192.168.1.66'})

def unique_error_ips(logs: Iterable[dict]) -> set[str]:
    """Ensemble des IPs ayant généré au moins une erreur."""

def ip_set_analysis(active_ips: set[str]) -> dict[str, set[str]]:
    """Retourne union, intersection, différence, diff symétrique avec
BLACKLIST."""
    return {
        'blacklisted_active': active_ips & BLACKLIST,
        'all_known': active_ips | BLACKLIST,
        'clean_active': active_ips - BLACKLIST,
        'exclusives': active_ips ^ BLACKLIST,
    }

def count_by_category(logs: Iterable[dict]) -> defaultdict[str, int]:
    """defaultdict(int) pour compter par level ou par user_id."""

def detect_bruteforce(logs: Iterable[dict], threshold: int = 5) -> list[str]:
    """Détecte N échecs de connexion consécutifs : while + for +
break/continue/else."""
```

Exigences :

- Utiliser `set`, `frozenset`, opérateurs `|`, `&`, `-`, `^`.
- Utiliser `collections.defaultdict`.
- Dans `detect_bruteforce`, **exploiter la clause `else` d'une boucle** (c'est explicitement noté au barème).

Module 4 — Rule Engine & Introspection (Dev C, 3 pts)

Fonctions clés attendues :

```
def apply_security_rules(*args: Callable, **kwargs) -> list[dict]:
    """Applique un nombre variable de règles (fonctions) avec config
dynamique.

    Exemple d'appel :
        apply_security_rules(check_bruteforce, check_blacklist,
                             threshold=5, mode='strict')
    """

def audit_runtime_state(tag: str = 'audit') -> dict:
    """Inspecte globals() et locals() et exporte un rapport des variables
chargées."""
    return {
        'tag': tag,
        'globals_keys': list(globals().keys()),
        'locals_snapshot': dict(locals()),
    }
```

Exigences :

- Les règles sont **des fonctions passées en argument** (first-class functions).
- `*args` ET `**kwargs` doivent être utilisés dans la même fonction maîtresse.
- `globals()` ET `locals()` doivent apparaître dans `audit_runtime_state`.

5. Planning sur 4 semaines

Semaine 1 — Setup & fondations

- **J1-J2 (en commun)** : création du repo Git, du `venv`, de l'arborescence complète (dossiers vides + `__init__.py`), du `.gitignore`, du `README.md` initial, choix des outils (`black`, `ruff`, `pytest`, `matplotlib`, `rich` pour la CLI).
- **J3-J5** : chacun ouvre une branche `feat/moduleX` et code les **signatures** de ses fonctions + docstrings + 1 test pytest qui passe.
- **En parallèle (Dev A)** : script de génération d'un fichier de logs synthétique ~100 Mo dans `data/`.
- **Checkpoint fin S1 (visio 30 min)** : chaque dev démontre que son module est importable et que `pytest` passe.

Semaine 2 — Modules 1 & 2 complets

- **Dev A** : finit `read_log_file` + `read_multiple_sources` + `parse_line` + tuples immuables. Preuve de non-saturation RAM avec `tracemalloc`.

- **Dev B** : finit `filter_invalid`, `normalize`, `route_log` (match-case avec 3+ cas).
- **Dev C** : avance sur `unique_error_ips`, `ip_set_analysis`, `count_by_category`.
- **Checkpoint fin S2** : `main.py` exécute lecture → parse → filter → map → affichage. Démo en live entre vous trois.

Semaine 3 — Modules 3 & 4 + intégration

- **Dev C** : finit `detect_bruteforce` (avec `else` sur boucle), `apply_security_rules`, `audit_runtime_state`.
- **Dev B** : intègre le rule engine dans `main.py` (option CLI `--rules`).
- **Dev A** : optimise la lecture, documente la consommation mémoire, génère un jeu de données avec attaques simulées.
- **Checkpoint fin S3** : pipeline complet end-to-end fonctionnel sur `sample_large.log`.

Semaine 4 — Finitions & livraison

- **Tous** : tests unitaires de chaque module (couverture cible : > 70 %).
- **Tous** : relecture croisée PEP8, type hints, gestion d'exceptions, présence de la clause `else` des boucles.
- **Dev C** : rédaction du `notebooks/demo_rapport.ipynb` avec graphes (top IPs en erreur, timeline des sévérités, courbe de détection brute-force).
- **Dev B** : `README.md` complet + exemples de commandes CLI.
- **Dev A** : `pip freeze > requirements.txt` final, tag `v1.0.0` sur Git.
- **Veille soutenance** : répétition de la démo (chacun son module) + relecture du barème point par point.

6. Conventions de code et règles d'équipe

Git

- Branche `main` protégée : interdit de push directement dessus.
- Nommage des branches : `feat/module-ingestion`, `fix/parser-timestamp`, `docs/readme`.
- Chaque feature → **Pull Request** avec au moins **1 relecteur** (un des deux autres devs).
- Commits clairs en français ou anglais, format court : `feat(ingestion): add yield from reader`.
- `.gitignore` inclut : `venv/`, `__pycache__/`, `*.pyc`, `.pytest_cache/`, `data/*.log` (sauf un petit échantillon).

Qualité code

- **PEP8** vérifié par `ruff` (lancer `ruff check src/` avant chaque PR).
- Formatage automatique : `black src/ tests/`.
- **Type hints** partout (`def foo(x: int) -> str:`).
- Docstrings courtes sur chaque fonction publique.
- Gestion d'exceptions explicite : pas de `except:` nu, toujours `except SpecificError`.

Communication

- Daily async de 15 min (WhatsApp / Discord), 3 fois par semaine : *fait / bloqué sur / à faire*.
- Checkpoint fin de semaine en visio (30 min min.).
- Un canal partagé pour les décisions techniques (éviter les décisions en DM privé).

Pièges à éviter (impacts barème)

- ❌ Remplacer `yield` par une liste chargée en mémoire → perte potentielle **4 pts**.
- ❌ Oublier la clause `else` sur une boucle de détection → perte sur **Qualité du code** (3 pts).
- ❌ Utiliser `list` au lieu de `frozenset` pour la blacklist → perte sur **Structures de données** (4 pts).
- ❌ Installer des packages sans mettre à jour `requirements.txt` → perte sur **Livrables** (2 pts).
- ❌ Remplacer `match-case` par des `if/elif` → perte sur **Structures de données**.
- ❌ Code logique dans les notebooks au lieu des `.py` → perte sur **Livrables**.

7. Livrables finaux et barème (20 points)

Livrables attendus (dépôt Git)

1. Arborescence Python structurée en modules (voir § 3).
2. `requirements.txt` fonctionnel (test : créer un `venv` neuf, `pip install -r requirements.txt`, lancer `main.py`).
3. `main.py` de démonstration exécutant l'ensemble du pipeline sur `data/sample_large.log`.
4. `notebooks/demo_rapport.ipynb` ou PDF illustrant le fonctionnement (graphiques + métriques).
5. `README.md` : installation, commandes CLI, explication rapide des modules.

Correspondance barème → responsable principal

Critère	Points	Responsable principal	Points de vigilance
Respect Environnement & Livrables	2	Tous (Dev A sur `requirements.txt`)	`venv` propre, pas de biblio non déclarée
Gestion mémoire & Générateurs	4	**Dev A**	`yield` + `yield from` démontrables, `tracemalloc` dans le rapport
Structures de données complexes	4	**Dev C** (+ Dev A pour tuples, Dev B pour match-case)	`set`, `frozenset`, `defaultdict`, `tuple`, `match-case` tous présents
Programmation Fonctionnelle	4	**Dev B**	`map`, `filter`, `lambda`, fonctions passées en argument
Moteur de Règles & Introspection	3	**Dev C**	`*args`, `**kwargs`, `globals()`, `locals()`
Qualité du code & Robustesse	3	Tous (Dev B coordonne)	PEP8, type hints, exceptions, clause `else` des boucles

Total : 20 points — chaque dev a au moins un critère dont il est principal responsable.

8. Annexes

A. Commandes de démarrage (Windows / PowerShell)

```
# Cloner le repo
git clone <url-du-repo> projetspython
cd projetspython

# Créer et activer le venv
python -m venv venv
.\venv\Scripts\Activate.ps1

# Installer les dépendances
pip install -r requirements.txt
```

```
# Lancer la démo
python main.py --input data/sample_large.log --rules bruteforce,blacklist

# Lancer les tests
pytest -v

# Vérifier la qualité
ruff check src/
black --check src/ tests/
```

B. Format d'une ligne de log (exemple)

```
2026-11-04T18:32:11Z | ERROR | 192.168.1.42 | user=admin |
{"action":"login","status":"failed"}
2026-11-04T18:32:12Z | INFO  | 10.0.0.15   | user=alice |
{"action":"view","resource":"/home"}
2026-11-04T18:32:14Z | CRITICAL | 192.168.1.66 | user=root |
{"action":"login","status":"failed"}
```

C. Checklist avant soutenance

- [ ] `pip freeze > requirements.txt` exécuté depuis le venv.
- [ ] Le repo clôné à neuf s'installe et s'exécute en < 3 commandes.
- [ ] `yield` et `yield from` grep-ables dans `src/ingestion/`.
- [ ] `filter(`, `map(`, `lambda` grep-ables dans `src/pipeline/`.
- [ ] `match` + au moins 3 `case` dans `src/pipeline/router.py`.
- [ ] `frozenset`, `set`, `defaultdict`, `tuple` tous présents dans `src/analytics/`.
- [ ] Au moins une boucle avec clause `else` dans `src/analytics/bruteforce.py`.
- [ ] `*args`, `**kwargs`, `globals()`, `locals()` dans `src/engine/`.
- [ ] Le notebook de démo s'exécute de haut en bas sans erreur.
- [ ] `pytest` passe au vert.
- [ ] README à jour avec capture ou gif de la démo.
- [ ] Tag Git `v1.0.0` posé.

D. Packages préconisés pour `requirements.txt`

```
matplotlib>=3.8
pytest>=8.0
black>=24.0
ruff>=0.4
rich>=13.0        # affichage CLI coloré
jupyter>=1.0      # pour le notebook de démo
```